SIMATS ENGINEERING

C03-ASSESSMENT TOOL 1

Experiments

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1110
Name: V V NAVEEN KUMAR
REG NO: 192321018

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models**.** *(BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:40**

1. Develop a system using UML for implementing the Book Bank process. It should verify the details of the student by the central computer. The details regarding the student will be provided to the central computer through the administrator in the book bank and the computer will verify the details of the student and provide approval to the office. Then the books that are needed by the student will issue from the office to him.

2. Develop a system using UML for implementing the Exam Registration Portal. It should verify the details of the candidate by the central computer. The details regarding the candidate will be provided to the central computer through the administrator and the computer will verify the details of the candidate and provide approval. Then the hall ticket will be issued from the office to the candidate.

3. Develop a system using UML for implementing Stock Maintenance System. In this system, the customer can place orders and purchase items with the aid of the stock dealer and central stock system. These orders are verified and the items are delivered to the customer. Then the stock details are to be updated accordingly.

4. Develop a system using UML for implementing Online Course Reservation System. This system is organized by the central management system. The student first browses and selects the desired course of their choice. The university then checks the availability of the seat if it is available the student is enrolled for the course.

Code of Conduct:
I V V Naveen Kumar(192321018) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student:
naveenkumarvv

RUBRICS FOR CALCULATION:

Experiments

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Question 1: Book Bank Process

Problem Statement

Develop a system using UML for implementing the Book Bank process. It should verify the details of the student by the central computer. The details regarding the student will be provided to the central computer through the administrator in the book bank and the computer will verify the details of the student and provide approval to the office. Then the books that are needed by the student will be issued from the office to him.

Section A — Understanding of Concepts (Requirement Analysis)

The Book Bank process is a verification-and-issue workflow involving four key roles. The Student initiates the process by providing personal and academic details. The Administrator acts as an intermediary who forwards these details to the Central Computer, which performs verification against the institutional database. Once verified, the Central Computer communicates approval to the Office, which is responsible for the physical issuing of books and maintaining stock records. Analyzing this narrative surfaces the core object-oriented elements — actors, responsibilities, and the objects (Student, Book, BookRecord) that carry state through the workflow.

Actors Identified	Use Cases Identified
Student	Submit Student Details
Administrator	Verify Student Details
Central Computer	Approve Request
Office Staff	Request / Issue Books
	Update Stock Record

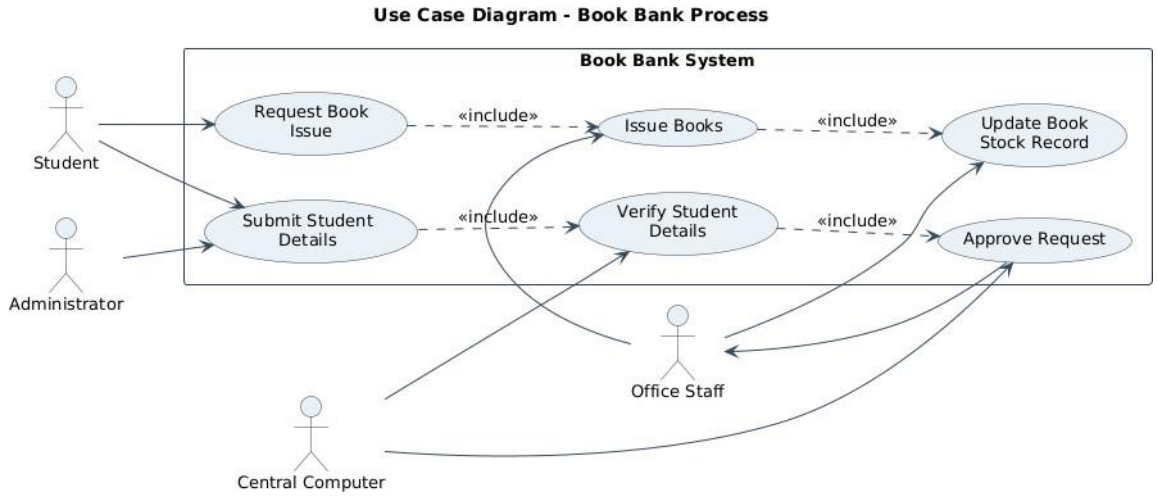
Section B — Experimental Design (UML Diagrams)

Figure 1.1 — Use Case Diagram

The use case diagram captures the four actors and the services the system exposes to them. Verification (UC2) and Approval (UC3) are modelled as included sub-flows of detail submission, since they always execute as part of that transaction, reflecting the mandatory verification requirement in the problem statement.

Figure 1.1: Use Case Diagram — Book Bank Process

Figure 1.2 — Class Diagram



The class diagram identifies five core classes. Student and Book are the primary domain entities; Administrator, CentralComputer and Office model the three-stage responsibility chain (forward → verify → issue) described in the problem statement. BookRecord captures the transactional issue/return state, decoupling the act of issuing from the static Book catalogue entry.

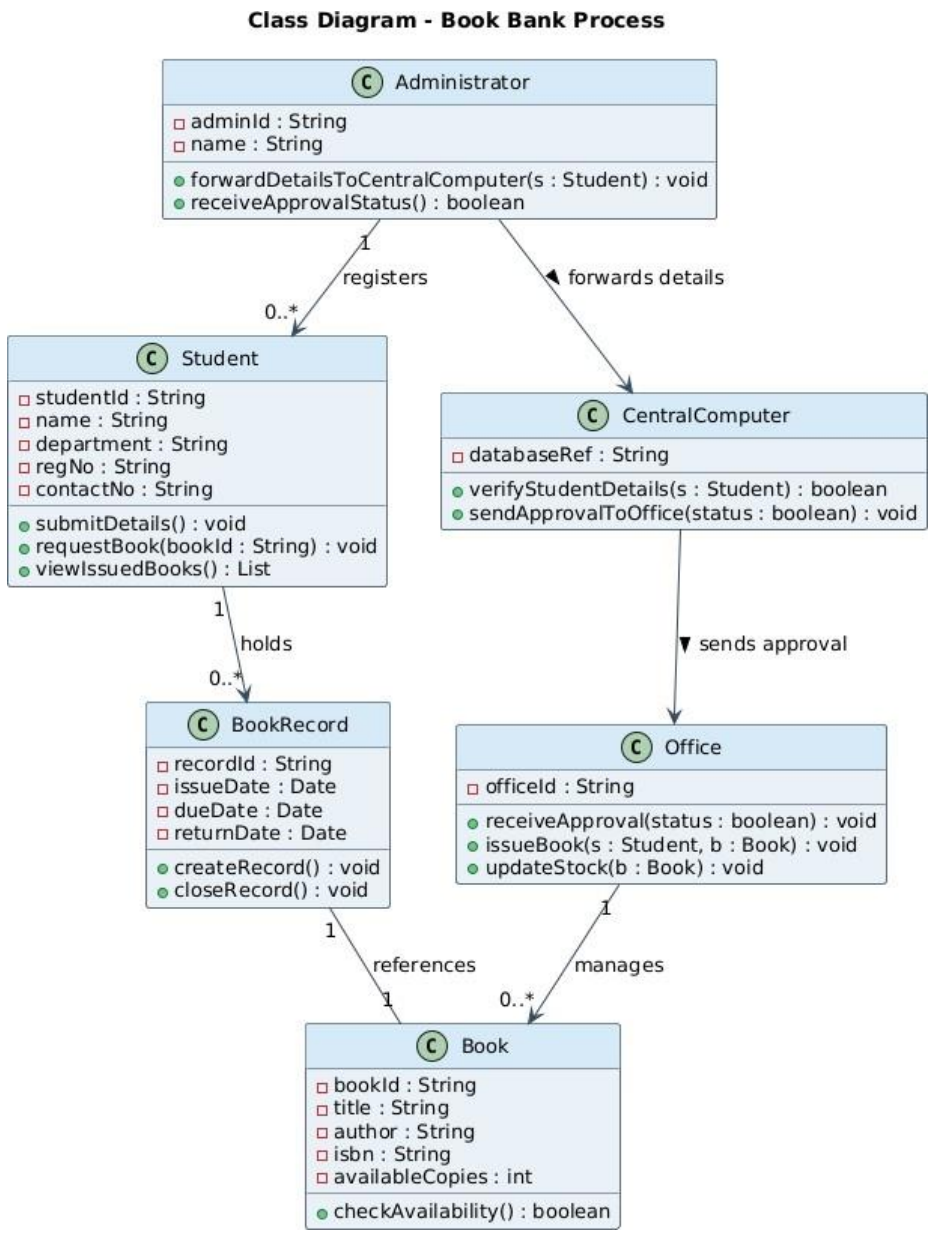


Figure 1.2: Class Diagram — Book Bank Process

Class	Key Attributes	Key Operations
Student	studentId, name, department, regNo	submitDetails(), requestBook(), viewIssuedBooks()
Administrator	adminId, name	forwardDetailsToCentralComputer(), receiveApprovalStatus()
CentralComputer	databaseRef	verifyStudentDetails(), sendApprovalToOffice()
Office	officeId	receiveApproval(), issueBook(), updateStock()
Book / BookRecord	bookId, title, availableCopies / issueDate, dueDate	checkAvailability() / createRecord(), closeRecord()

Figure 1.3 — Sequence Diagram

The sequence diagram traces one complete transaction over time: the student-to-administrator handoff, the administrator-to-central-computer verification call, and the conditional (alt) branch that separates the approval path from the rejection path — directly reflecting the 'verify... and provide approval' requirement.

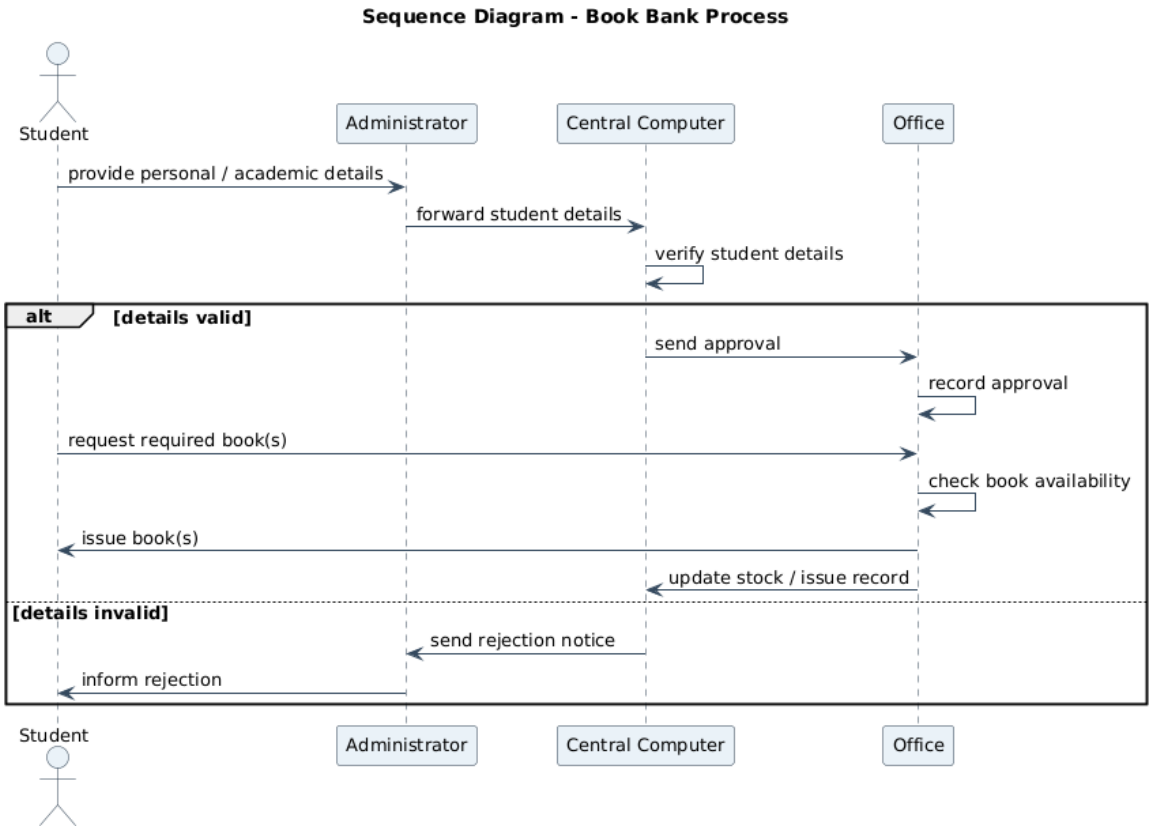


Figure 1.3: Sequence Diagram — Book Bank Process

Figure 1.4 — Activity Diagram

The activity diagram expresses the same logic as a control-flow chart with two decision points (Details Valid?, Book Available?), making the branching business rules explicit and easy to validate against the requirement text.

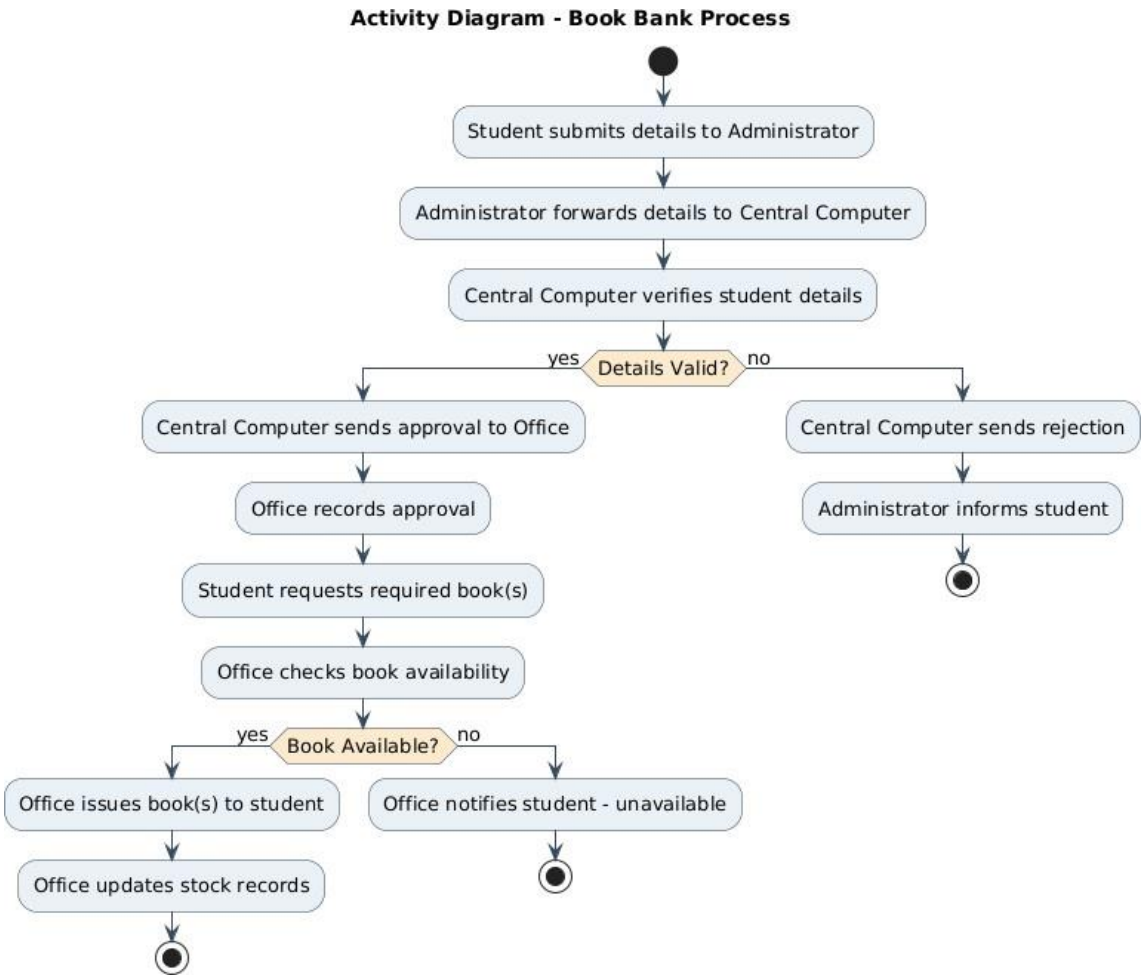


Figure 1.4: Activity Diagram — Book Bank Process

Section C — Use of Tools

All diagrams were modelled in PlantUML, a text-driven UML 2.5 modelling tool, and rendered to PNG for embedding. PlantUML was chosen because it produces standards-compliant notation (actors, association/include/extend stereotypes, multiplicities, activity swim-decisions) directly from a readable textual specification, which keeps the model easy to version and modify — the same benefit that makes UML tools preferable to freehand drawing in a real design workflow.

Section D — Analysis of Results

- Traceability: every actor named in the problem statement (student, administrator, central computer, office) appears consistently across all four diagrams.
- Consistency check: the <<include>> relationships in the use case diagram (Submit → Verify → Approve) match the sequential message order in the sequence diagram.
- The class diagram's associations (Administrator → CentralComputer → Office) mirror the three-stage escalation path described narratively, confirming the object model is faithful to requirements.
-

- The activity diagram's two decision branches (validity, availability) show the system correctly handles both the happy path and the rejection / unavailability exceptions implied by 'verify... and provide approval'.

Cross-validating the four views confirms the design is internally consistent: no actor, message, or class appears in one diagram without a corresponding element in the others, satisfying the requirement that the central computer gate book issuance behind a verification-and-approval step.

Section E — Documentation & Traceability

Each diagram is numbered and captioned (Figures 1.1–1.4) for direct reference. The class summary table above documents attributes and operations that are compressed in the diagram for readability, and the rubric-mapping table below shows exactly where each assessment criterion is satisfied in this answer.

Question 2: Exam Registration Portal

Problem Statement

Develop a system using UML for implementing the Exam Registration Portal. It should verify the details of the candidate by the central computer. The details regarding the candidate will be provided to the central computer through the administrator and the computer will verify the details of the candidate and provide approval. Then the hall ticket will be issued from the office to the candidate.

Section A — Understanding of Concepts (Requirement Analysis)

The Exam Registration Portal follows the same verify-then-release pattern as the Book Bank system but substitutes 'candidate' for 'student' and 'hall ticket' for 'book'. The Candidate submits registration details; the Administrator relays them to the Central Computer for verification; on approval, the Office generates and issues a hall ticket. An additional real-world step — fee payment — is included as a precondition for hall-ticket generation, since exam registration is not considered complete without it, giving the model one more decision point than a pure verification flow.

Actors Identified	Use Cases Identified
Candidate	Submit Candidate Details
Administrator	Verify Candidate Details
Central Computer	Approve Registration
Office Staff	Pay Exam Fee
	Generate & Issue Hall Ticket

Section B — Experimental Design (UML Diagrams)

Figure 2.1 — Use Case Diagram

As with Q1, verification and approval are modelled as included sub-flows of detail submission. 'Pay Exam Fee' is a separate use case performed directly by the candidate, feeding into hall-ticket generation, reflecting that payment and identity-verification are independent preconditions.

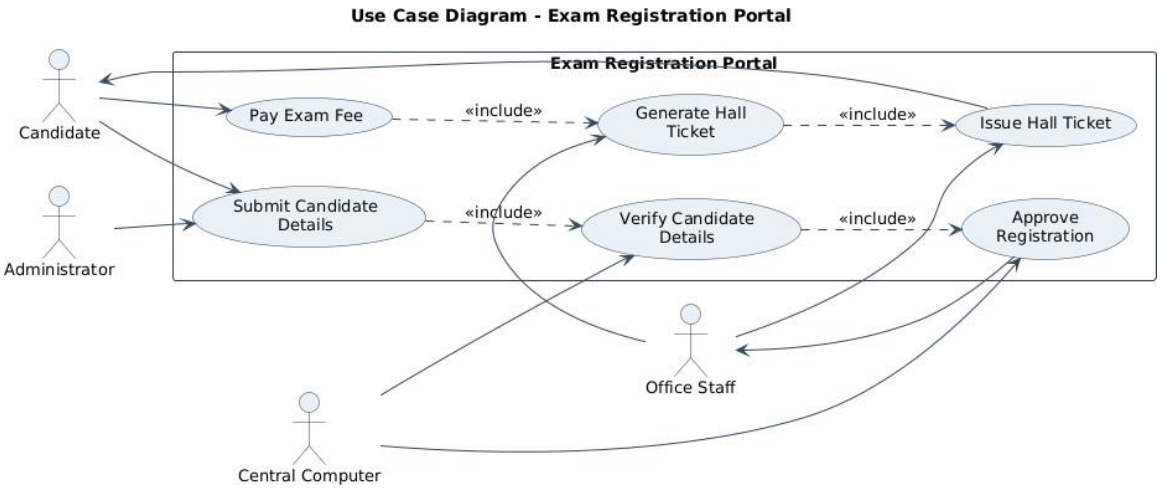


Figure 2.1: Use Case Diagram — Exam Registration Portal

Figure 2.2 — Class Diagram

ExamRegistration is introduced as a transactional class linking a Candidate to a specific course/exam and tracking fee status, distinct from the static Candidate profile. HallTicket is a generated artifact produced once both verification and payment succeed.

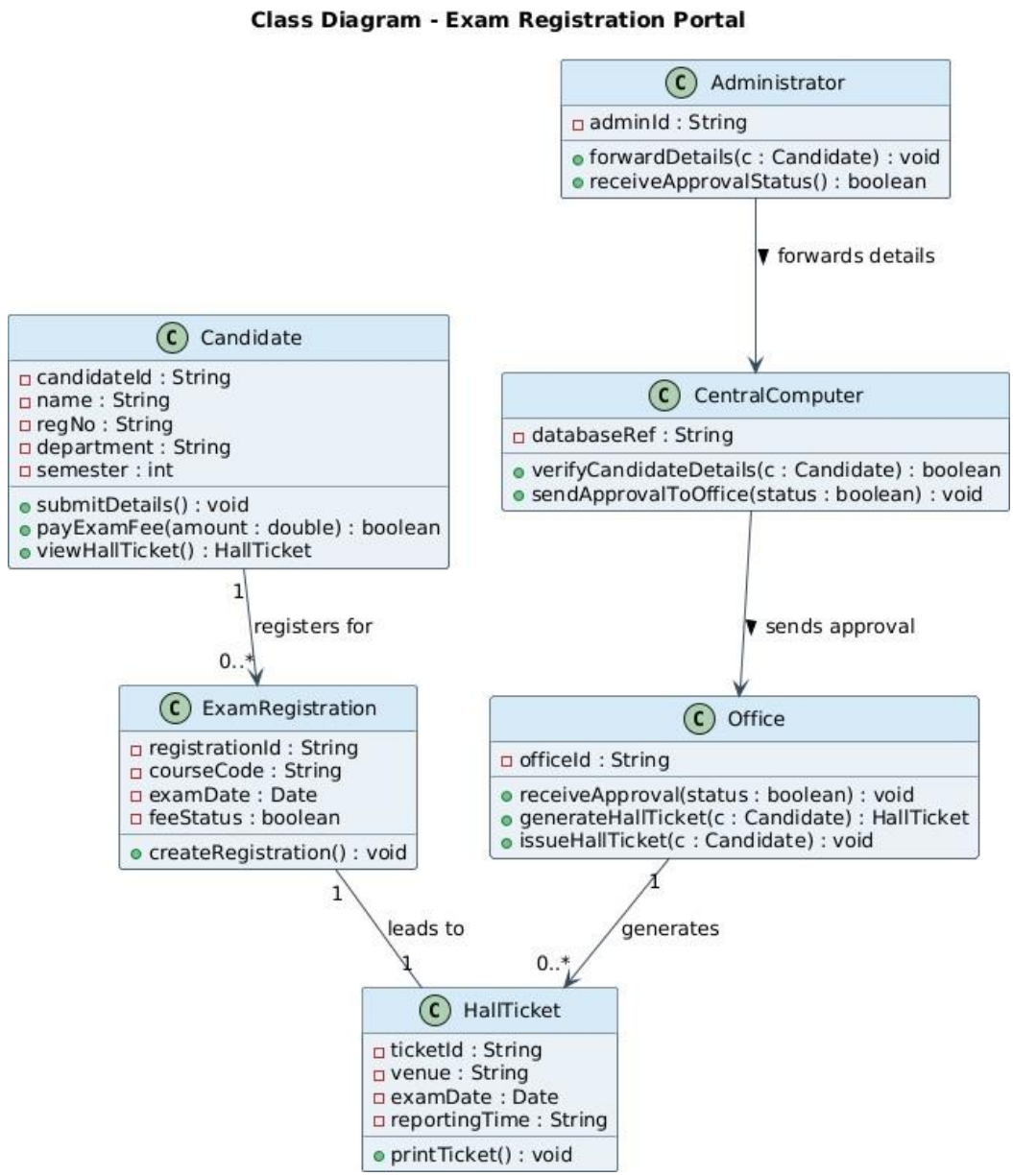


Figure 2.2: Class Diagram — Exam Registration Portal

Class	Key Attributes	Key Operations
Candidate	candidateId, name, regNo, semester	submitDetails(), payExamFee(), viewHallTicket()
Administrator	adminId	forwardDetails(), receiveApprovalStatus()
CentralComputer	databaseRef	verifyCandidateDetails(), sendApprovalToOffice()
Office	officeId	receiveApproval(), generateHallTicket(), issueHallTicket()

ExamRegistration / HallTicket	courseCode, examDate, feeStatus / venue, reportingTime	createRegistration() / printTicket()

Figure 2.3 — Sequence Diagram

The sequence diagram shows verification occurring first (candidate → administrator → central computer), followed by a second, independent gate — fee payment — before the office generates and issues the hall ticket, giving a two-condition happy path that mirrors real examination-cell procedure.

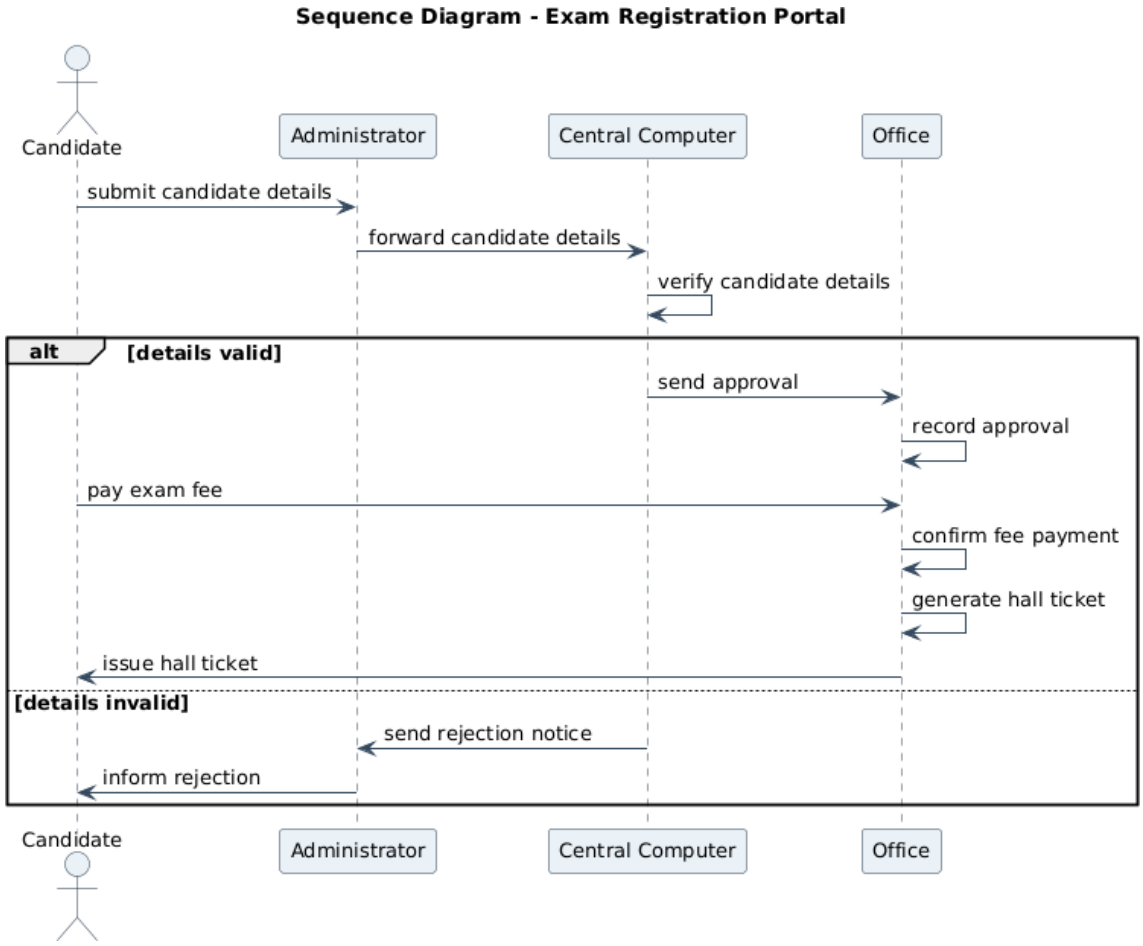


Figure 2.3: Sequence Diagram — Exam Registration Portal

Figure 2.4 — Activity Diagram

Two sequential decisions (Details Valid?, Payment Successful?) are shown, each with its own rejection/failure branch, so the diagram documents all four possible outcomes of the registration attempt.

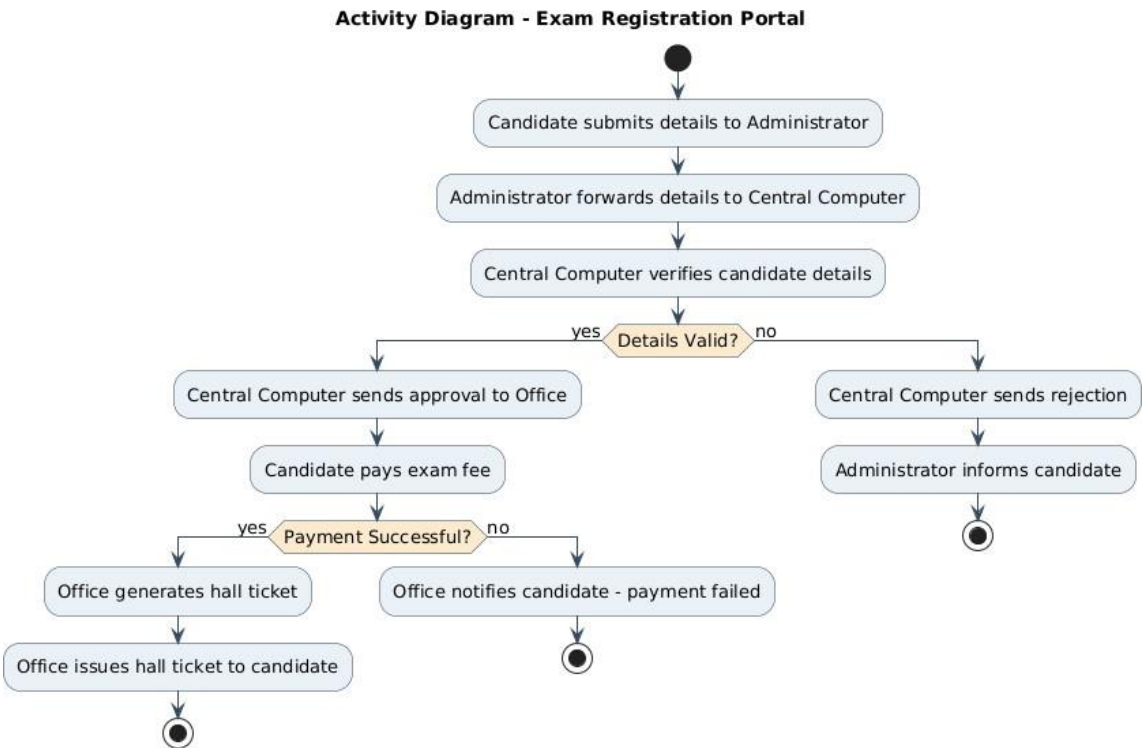


Figure 2.4: Activity Diagram — Exam Registration Portal

Section C — Use of Tools

The same PlantUML toolchain used in Question 1 was reused here for notational consistency across the assessment, again rendered to PNG. Reusing one tool across all four questions also demonstrates consistent, correct application of UML syntax rather than one-off diagrams.

Section D — Analysis of Results

- The central-computer verification gate from the problem statement is preserved identically to Q1, confirming the reusable nature of the 'verify-then-approve' pattern across institutional workflows.
- The additional fee-payment step is consistently represented in the use case (separate use case), class (feeStatus attribute), sequence (extra alt block) and activity (extra decision diamond) diagrams — showing the four views stay synchronized even as complexity increases.
- No hall ticket can be generated in any diagram without both verification and payment succeeding, correctly enforcing the two preconditions implied by the narrative.

The model satisfies the stated requirement — central computer verification gating office-side issuance — while realistically extending it with a payment precondition, and all four diagrams remain mutually consistent under that extension.

Section E — Documentation & Traceability

Figures 2.1–2.4 are numbered and captioned for direct reference; the class summary and rubric-mapping tables below complete the documentation expected for this criterion.

Question 3: Stock Maintenance System

Problem Statement

Develop a system using UML for implementing Stock Maintenance System. In this system, the customer can place orders and purchase items with the aid of the stock dealer and central stock system. These orders are verified and the items are delivered to the customer. Then the stock details are to be updated accordingly.

Section A — Understanding of Concepts (Requirement Analysis)

This system differs structurally from Q1/Q2: there is no administrator-relayed detail-verification step; instead the Customer interacts directly with the Stock Dealer, who checks item availability against the Central Stock System before delivering goods and updating inventory. The core object-oriented challenge is separating the transactional Order from the persistent Item/stock record, and ensuring the stock count is only updated after delivery is confirmed.

Actors Identified	Use Cases Identified
Customer	Place Order
Stock Dealer	Verify Order
Central Stock System	Check Stock Availability
	Deliver Items
	Update Stock Details
	Generate Invoice

Section B — Experimental Design (UML Diagrams)

Figure 3.1 — Use Case Diagram

Order verification and stock-availability checking are modelled as included sub-flows of order placement, while invoice generation is included under item delivery, reflecting that an invoice is only produced once goods are actually delivered.

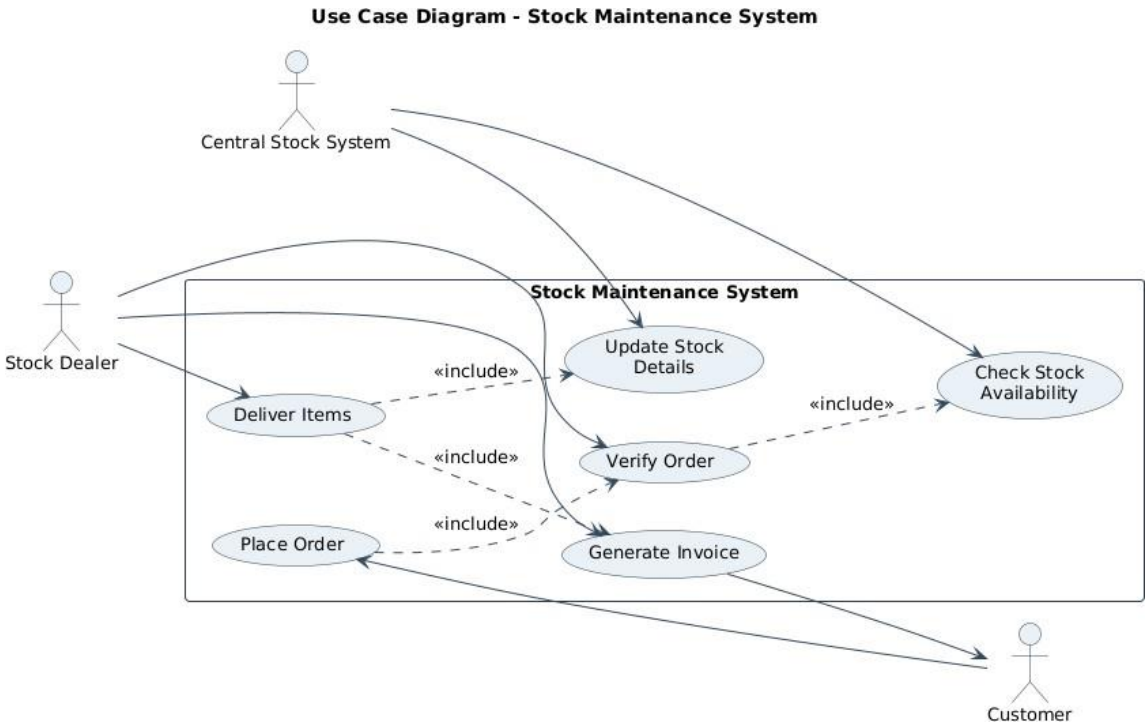


Figure 3.1: Use Case Diagram — Stock Maintenance System

Figure 3.2 — Class Diagram

Order aggregates one or more Item lines and computes a total; CentralStockSystem is kept separate from StockDealer because, per the narrative, the dealer 'checks with the aid of' the central system rather than owning the stock data directly — a deliberate separation of concerns between the sales role and the inventory-of-record.

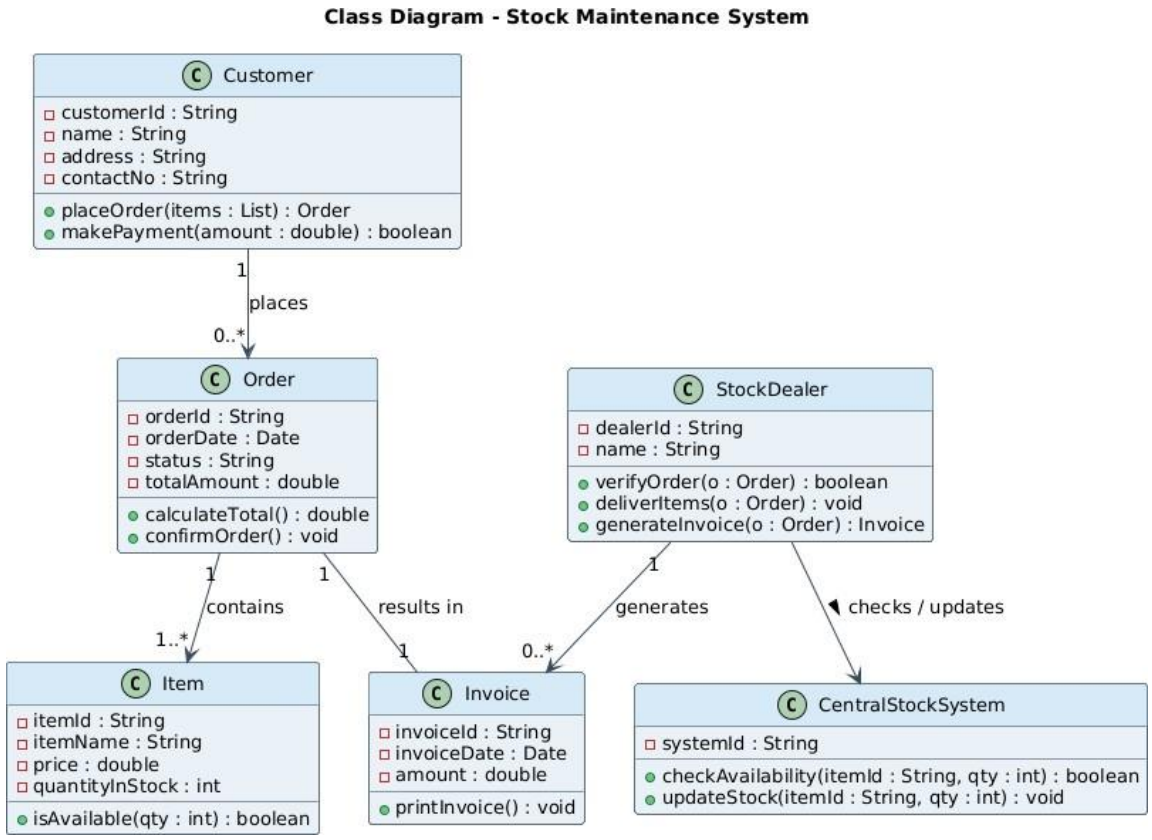


Figure 3.2: Class Diagram — Stock Maintenance System

Class	Key Attributes	Key Operations
Customer	customerId, name, address	placeOrder(), makePayment()
StockDealer	dealerId, name	verifyOrder(), deliverItems(), generateInvoice()
CentralStockSystem	systemId	checkAvailability(), updateStock()
Order	orderId, orderDate, status, totalAmount	calculateTotal(), confirmOrder()
Item / Invoice	itemId, price, quantityInStock / amount	isAvailable() / printInvoice()

Figure 3.3 — Sequence Diagram

The sequence diagram shows the Stock Dealer as the mediator between Customer and CentralStockSystem: the dealer verifies the order, then queries the central system for availability before committing to delivery — an explicit two-party check rather than a single verifying authority, matching the narrative's 'with the aid of the stock dealer and central stock system'.

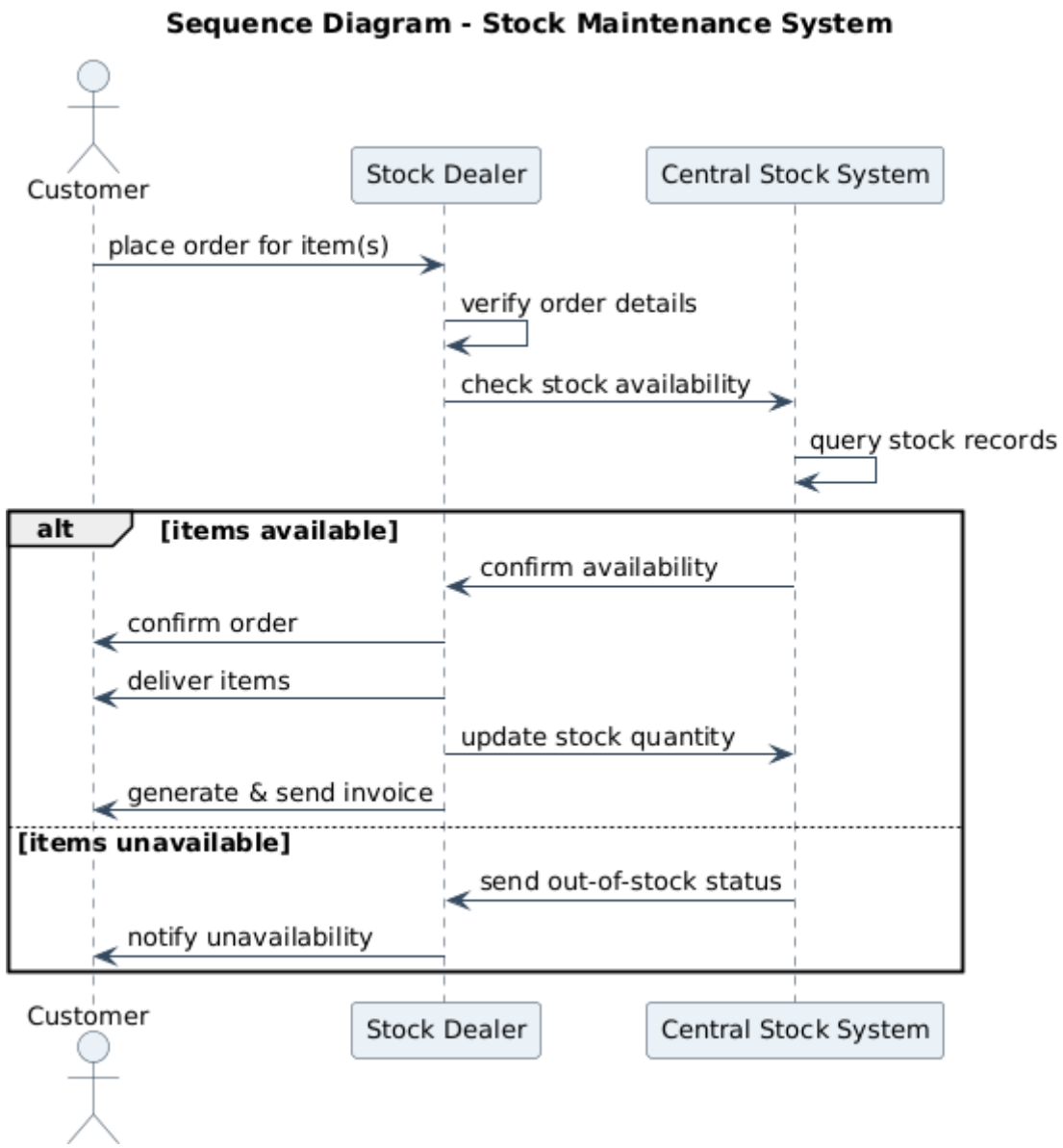


Figure 3.3: Sequence Diagram — Stock Maintenance System

Figure 3.4 — Activity Diagram

A single decision point (Items Available?) branches into delivery-and-stock-update versus a customer notification of unavailability, keeping the control flow tight and directly traceable to the two outcomes described in the problem statement.

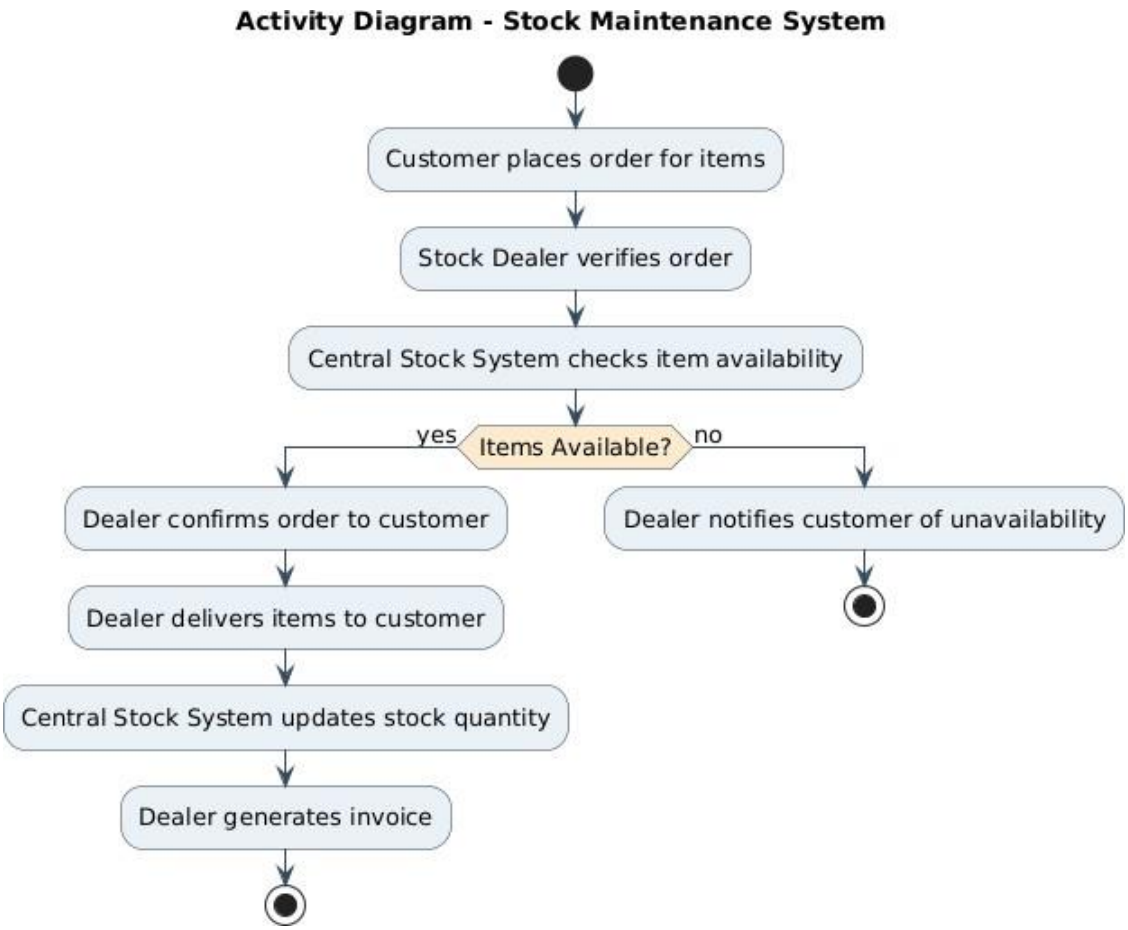


Figure 3.4: Activity Diagram — Stock Maintenance System

Section C — Use of Tools

Diagrams were again produced in PlantUML for notational consistency with Questions 1, 2 and 4, using UML association, multiplicity and stereotype conventions appropriate to an inventory/order-management domain (e.g., 1..* composition between Order and Item).

Section D — Analysis of Results

- Delivery and stock-update are ordered correctly across the sequence and activity diagrams — stock is only decremented after delivery is confirmed, avoiding the classic inventory bug of reserving stock before a sale is finalized.
- The class diagram's Order–Item composition (1 to 1..*) matches the plural 'items' language in the problem statement, rather than modelling a single-item order.
- CentralStockSystem's role is consistently limited to availability-checking and stock updates across every diagram, with delivery and invoicing kept as StockDealer responsibilities — a clean separation of concerns.

The four diagrams jointly validate a correct order-to-delivery-to-restock pipeline in which stock figures are only ever mutated after a successful delivery, satisfying the 'stock details are to be updated accordingly' requirement.

Section E — Documentation & Traceability

Figures 3.1–3.4 and the accompanying class summary/rubric tables provide the structured documentation expected for this section, consistent in format with Questions 1, 2 and 4.

Rubric-to-Content Mapping (10 marks)

Rubric Criterion (Weightage)	Marks (/10)	Addressed In
Understanding of Concepts (25%)	2.5	Section A – Requirement Analysis
Experimental Design (30%)	3.0	Section B – UML Diagrams
Use of Tools (20%)	2.0	Section C – Tool & Notation Used
Analysis of Results (15%)	1.5	Section D – Workflow Analysis
Documentation (10%)	1.0	Section E – Class Summary & Traceability

Question 4: Online Course Reservation System

Problem Statement

Develop a system using UML for implementing Online Course Reservation System. This system is organized by the central management system. The student first browses and selects the desired course of their choice. The university then checks the availability of the seat; if it is available, the student is enrolled for the course.

Section A — Understanding of Concepts (Requirement Analysis)

This is the simplest workflow of the four: a single decision gate (seat availability) determines whether a browsing-and-selecting student becomes an enrolled student. The Central Management System coordinates between the Student-facing browsing interface and the University, which owns authoritative seat data and performs enrollment — a natural split between a presentation/coordination layer and a system-of-record.

Actors Identified	Use Cases Identified
Student	Browse Courses
Central Management System	Select Course
University	Check Seat Availability
	Enroll Student
	Confirm Reservation

Section B — Experimental Design (UML Diagrams)

Figure 4.1 — Use Case Diagram

Seat-availability checking is included under course selection (it always follows selection), while enrollment is modelled as an <<extend>> of the availability check — it only executes on the conditional path where a seat is actually available, which is the correct UML idiom for an optional, condition-triggered use case.

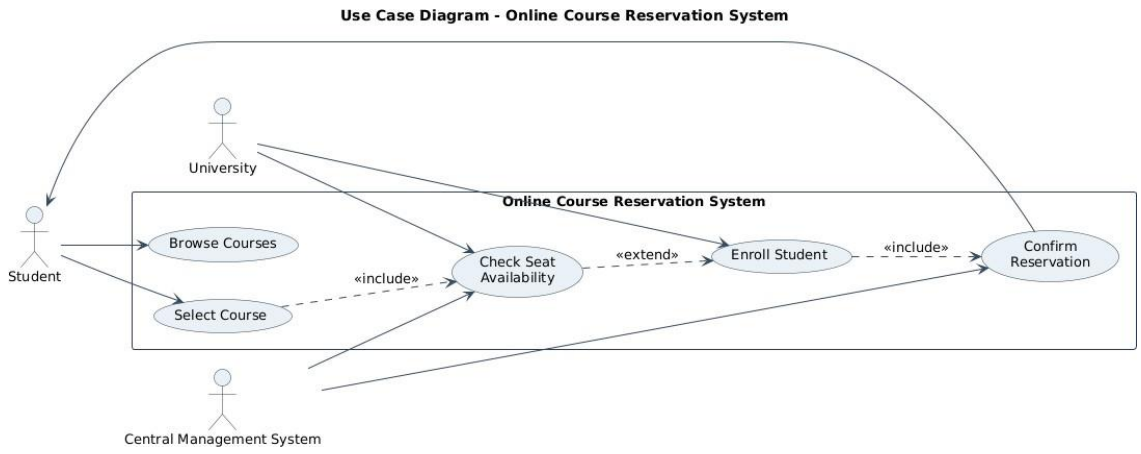


Figure 4.1: Use Case Diagram — Online Course Reservation System

Figure 4.2 — Class Diagram

Reservation is introduced as the join-class linking a Student to a Course once a seat has been confirmed, keeping the Course's own seat-count logic (isSeatAvailable, decrementSeat) independent of who is enrolling — a design that scales cleanly if multiple students attempt to reserve the same course concurrently.

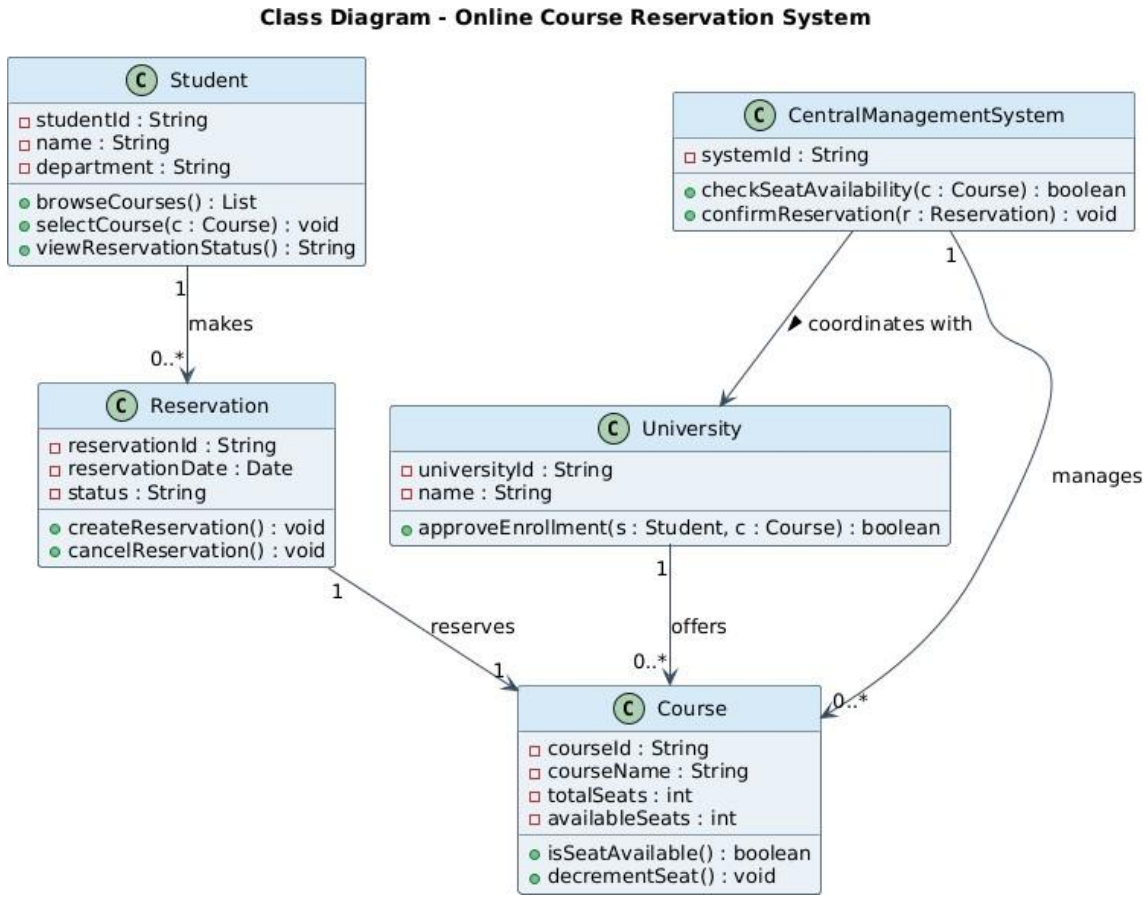


Figure 4.2: Class Diagram — Online Course Reservation System

Class	Key Attributes	Key Operations
Student	studentId, name, department	browseCourses(), selectCourse(), viewReservationStatus()
CentralManagementSystem	systemId	checkSeatAvailability(), confirmReservation()
University	universityId, name	approveEnrollment()
Course	courseId, courseName, totalSeats, availableSeats	isSeatAvailable(), decrementSeat()
Reservation	reservationId, reservationDate, status	createReservation(), cancelReservation()

Figure 4.3 — Sequence Diagram

The sequence diagram shows the Central Management System acting as broker: it forwards the availability check to the University and only proceeds to request enrollment on the success branch, directly implementing the 'university then checks... if available, the student is enrolled' logic from the problem statement.

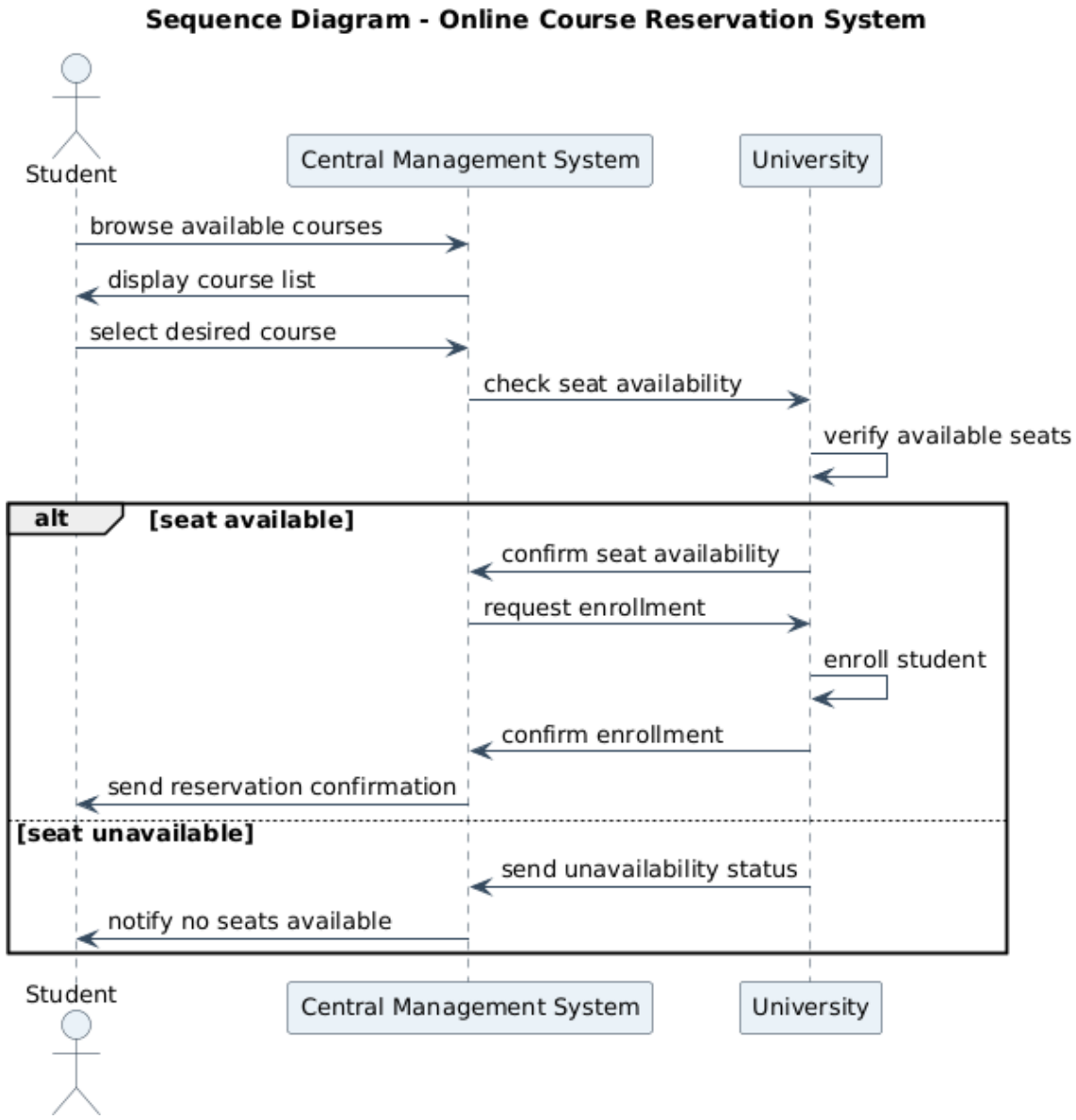


Figure 4.3: Sequence Diagram — Online Course Reservation System

Figure 4.4 — Activity Diagram

A single decision diamond (Seat Available?) cleanly separates the enrollment-and-confirmation path from the notify-unavailable path, matching the problem statement's simple two-outcome description.

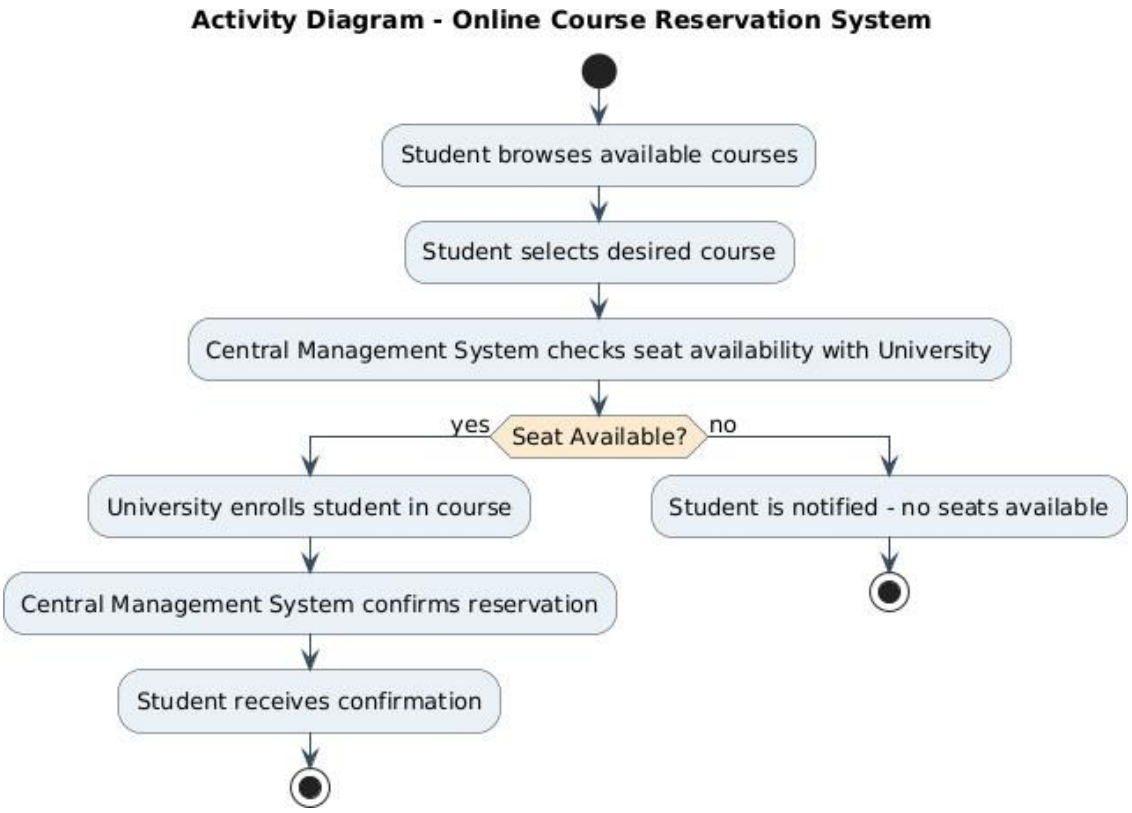


Figure 4.4: Activity Diagram — Online Course Reservation System

Section C — Use of Tools

Modelled in PlantUML using the same house style (colour palette, font, notation) as Questions 1–3, so that the full submission reads as one consistent design rather than four unrelated diagram sets.

Section D — Analysis of Results

- The <<extend>> relationship on Enroll Student in the use case diagram is the correct UML construct for a conditionally-triggered use case, and it is honoured consistently in the activity diagram's if/else branch.
- University, not the Central Management System, owns approveEnrollment() in the class diagram — matching the narrative's assignment of the enrollment decision to the university rather than the coordinating portal.
- The Course class's own decrementSeat() operation keeps seat-count mutation localized to the entity being reserved, reducing the risk of inconsistent seat counts if the system is extended to concurrent reservations.

All four views agree on a single-gate enrollment workflow — browse, select, check availability, enroll-or-notify — fully covering the requirement with no extraneous states or actors.